

Objets JavaScript

Dans ce chapitre, vous apprendrez à créer et à utiliser des objets en JavaScript.

Qu'est-ce qu'un objet ?

JavaScript est un langage basé sur des objets et en JavaScript presque tout est un objet ou agit comme un objet. Ainsi, pour travailler avec JavaScript de manière efficace et efficiente, nous devons comprendre comment les objets fonctionnent ainsi que comment créer vos propres objets et les utiliser.

Un objet JavaScript est juste une collection de valeurs nommées. Ces valeurs nommées sont généralement appelées propriétés de l'objet.

Objets, propriétés et méthodes de la vie réelle

Dans la vraie vie, une voiture est un objet. Une voiture a des propriétés comme le poids et la couleur, et des méthodes comme le démarrage et l'arrêt

Objet	Propriétés	Méthodes
	<code>car.name = Fiat</code>	<code>car.start()</code>
	<code>car.model = 500</code>	<code>car.drive()</code>
	<code>car.weight = 850kg</code>	<code>car.brake()</code>
	<code>car.color = blanc</code>	<code>car.stop()</code>

Toutes les voitures ont les mêmes propriétés , mais les valeurs des propriétés diffèrent d'une voiture à l'autre.

Toutes les voitures ont les mêmes méthodes , mais les méthodes sont exécutées à des moments différents .

Objets JavaScript

Vous avez déjà appris que les variables JavaScript sont des conteneurs pour les valeurs de données. Ce code attribue une valeur simple (Fiat) à une variable nommée car :

```
let car = "Fiat";
```

Les objets sont aussi des variables. Mais les objets peuvent contenir de nombreuses valeurs. Ce code attribue plusieurs valeurs (Fiat, 500, blanc) à une variable nommée car

```
const car = {type:"Fiat", model:"500", color:"white"};
```

Les valeurs sont écrites sous forme de paires **nom:valeur** (nom et valeur séparés par deux points).

Il est courant de déclarer des objets avec le mot-clé `const` .

Définition d'objet

Un objet peut être créé avec des accolades `{}` avec une liste facultative de propriétés. Une propriété est une paire "**clé** : **valeur**", où la clé (ou le nom de la propriété) est **toujours une chaîne**, et la valeur (ou la valeur de la propriété) **peut être n'importe quel type de données**, comme des chaînes, des nombres, des booléens ou des types de données complexes comme des tableaux, fonctions et autres objets. De plus, les propriétés avec des fonctions comme valeurs sont souvent appelées méthodes pour les distinguer des autres propriétés.

Vous définissez (et créez) un objet JavaScript avec un littéral d'objet :

```
const person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

Les espaces et les sauts de ligne ne sont pas importants. Une définition d'objet peut s'étendre sur plusieurs lignes :

```
const person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 50,  
  eyeColor: "blue"  
};
```

Propriétés de l'objet

Les paires **name:values** dans les objets JavaScript sont appelées **properties (propriétés)**

Propriété	Valeur de la propriété
firstName	John
lastName	Doe
age	50
eyeColor	blue

Accéder aux propriétés de l'objet

Vous pouvez accéder aux propriétés d'un objet de deux manières :

```
objectName.propertyName
```

Exemple :

```
person.lastName;
```

Ou bien

```
objectName["propertyName"]
```

Exemple :

```
person["lastName"];
```

Méthodes d'objet

Les objets peuvent aussi avoir des méthodes. Les méthodes sont des actions qui peuvent être effectuées sur des objets. Les méthodes sont stockées dans les propriétés en tant que définitions de fonction .

Propriété	Valeur Propriété
firstName	John
lastName	Doe
age	50
eyeColor	blue
fullName	function() {return this.firstName + " " + this.lastName;}

Une méthode est une fonction stockée en tant que propriété.

Exemple de méthode :

```
const person = {  
  firstName: "John",  
  lastName : "Doe",  
  id        : 5566,  
  fullName : function() {  
    return this.firstName + " " + this.lastName;  
  }  
};
```

Dans l'exemple ci-dessus, **this** fait référence à l'objet **person** .

Ex: *this.firstName* signifie la propriété **firstName** de **this** .

Ex: *this.firstName* signifie la propriété **firstName** de **person**

Le mot-clé **this**

En JavaScript, le mot-clé **this** fait référence à un objet.

Le choix de cet objet dépend de la façon dont il est invoqué (utilisé ou appelé).

Le mot-clé **this** fait référence à différents objets selon la façon dont il est utilisé :

Dans une méthode d'objet, **this** fait référence à l' **objet** .

Seul, **this** fait référence à l' **objet global** .

Dans une fonction, **this** fait référence à l' **objet global** .

Dans une fonction, en mode strict, **this** vaut undefined.

Dans un événement, **this** fait référence à l' **élément** qui a reçu l'événement.

Des méthodes telles que call(), apply() et bind() peuvent faire référence **this** à **n'importe quel objet** .

Noter: **this** n'est pas une variable. C'est un mot clé. Vous ne pouvez pas modifier la valeur de **this**.

Dans une définition de fonction, **this** fait référence au "propriétaire" de la fonction.

Dans l'exemple ci-dessus, **this** est l' objet personne qui "possède" la fonction **fullName**.

En d'autres termes, **this.firstName** désigne la propriété **firstName** de cet objet .

Accéder aux méthodes d'objet

Vous accédez à une méthode d'objet avec la syntaxe suivante :

```
objectName.methodName()
```

Exemple

```
person.fullName();
```

Si vous accédez à une méthode sans les parenthèses (), elle retournera la définition de la fonction :

Exemple

```
person.fullName;
```